

Traduce la jerga de *Web client and backend development frameworks* a español simple, con analogías. Búscalo cuando el profe suelte un término nuevo.

Busca un término... ej. CORS, JWT, ORM, hexagonal

Todas

I · Arquitecturas

II · APIs

III · Backend

IV · Frontend

V · Nube

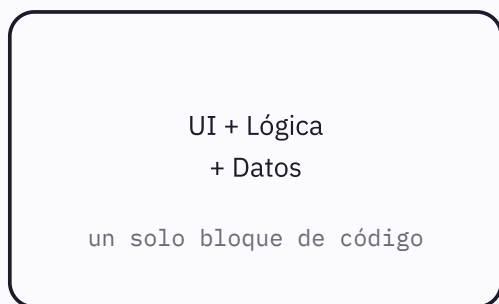
Stacks

UNIDAD I

Arquitecturas de desarrollo web

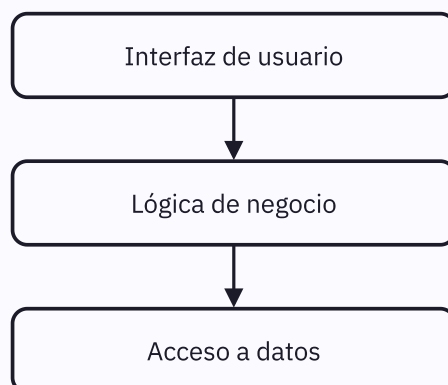
Lo que se espera que sepas hacer: diseñar una aplicación web con base en la arquitectura de microservicios.

Monolítica



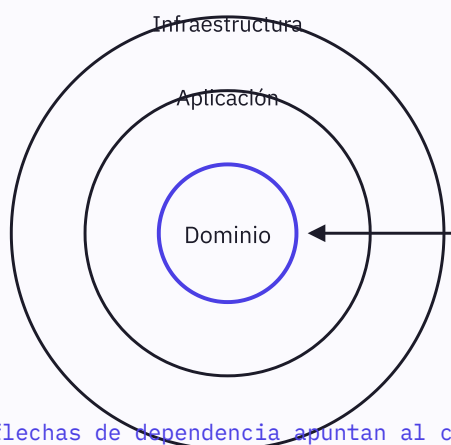
1 solo despliegue

N-capas



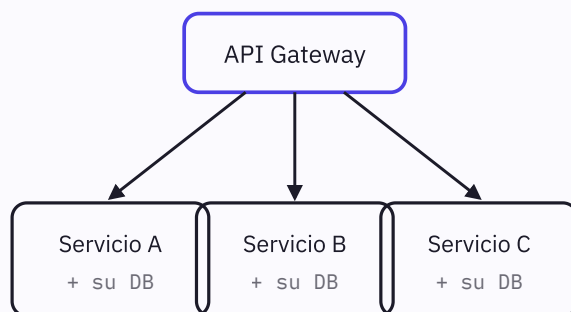
cada capa solo llama a la de abajo

Cebolla / Hexagonal



las flechas de dependencia apuntan al centro

Microservicios



cada uno se despliega solo

Las cuatro arquitecturas organizan el mismo código de forma distinta: un solo bloque, capas horizontales, anillos que protegen el dominio, o piezas independientes que se comunican por red.

Arquitectura monolítica

Todo el sistema (interfaz, lógica y datos) vive en un solo proyecto y se despliega como una sola pieza.

Piénsalo como: una navaja suiza soldada en un solo cuerpo — si se rompe una hoja, hay que reparar toda la navaja.

Arquitectura de n-capas

El código se separa en capas horizontales (interfaz, lógica de negocio, acceso a datos); cada una solo habla con la de al lado.

Piénsalo como: una fábrica en pisos — producción arriba, empaque en medio, bodega abajo; cada piso solo entrega al de junto.

Capa de acceso a datos

La única parte del sistema autorizada a hablar directamente con la base de datos.

Piénsalo como: el único empleado con llave de la bodega.

Capa de lógica de negocio

Donde viven las reglas y validaciones del negocio, sin saber cómo se ve la pantalla ni cómo se guardan los datos.

Piénsalo como: el cerebro de la operación — no le importa si el pedido llegó por app o por teléfono, solo aplica las reglas.

Arquitectura de cebolla

Onion / Clean Architecture

Capas concéntricas donde el dominio (reglas del negocio) está en el centro; todo lo demás depende hacia adentro, nunca al revés.

Piénsalo como: una cebolla real — puedes cambiar la cáscara (base de datos, framework) sin tocar el corazón.

Arquitectura hexagonal

puertos y adaptadores

Prima hermana de la cebolla: el dominio define "puertos" (contratos) y los "adaptadores" externos se conectan a esos puertos.

Piénsalo como: los enchufes de pared son el puerto; el adaptador de viaje cambia según el país, pero el aparato no se entera.

Arquitectura de microservicios

La app se parte en servicios pequeños e independientes, cada uno con su propia base de datos y despliegue, comunicados por red.

Piénsalo como: en vez de un restaurante con una cocina gigante, varios food trucks especializados que se coordinan.

Contenedor adelanto — Unidad V

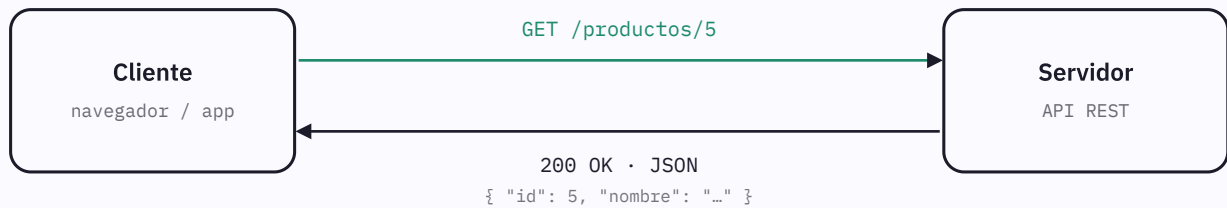
Una caja ligera que empaqueta una app con todo lo que necesita para correr igual en cualquier máquina.

Piénsalo como: una maleta que ya trae todo tuyo empackado, no depende de lo que haya en el hotel.

UNIDAD II

Desarrollo de APIs y servicios web

Lo que se espera que sepas hacer: desarrollar un sistema de servicios web con base en los principios REST y API RESTful.



El cliente pide un recurso con un verbo HTTP y una URI; el servidor responde con un código de estado y, casi siempre, datos en JSON.

Servicio web

Una funcionalidad que un sistema expone para que otro sistema (no una persona) la consuma por red.

Piénsalo como: una ventanilla de trámites, pero para programas.

SOAP

Protocolo antiguo y estricto para servicios web; usa XML obligatorio y reglas muy rígidas.

Piénsalo como: una carta formal con formato notarial.

REST

Estilo (no protocolo) para diseñar servicios web sobre HTTP, usando URIs para recursos y verbos para acciones. Es el estándar de facto hoy.

Piénsalo como: pedir en un restaurante con menú fijo — cada plato tiene un nombre y pides con verbos claros.

XML

Formato de intercambio de datos basado en etiquetas anidadas; verboso pero muy estructurado.

Piénsalo como: una carta con sobres dentro de sobres.

JSON

Formato de intercambio de datos ligero basado en pares llave-valor; el favorito de las APIs REST modernas.

Piénsalo como: una lista de compras con "producto: cantidad".

URI / Endpoint

La dirección única que identifica un recurso en el servidor (ej. /usuarios/5). El endpoint es el punto de entrada concreto de la API.

Piénsalo como: la dirección de una casa específica, no del barrio completo.

Verbo GET

Pide o lee un recurso sin modificarlo.

Piénsalo como: mirar el menú.

Verbo POST

Crea un recurso nuevo.

Piénsalo como: hacer un pedido nuevo.

Verbo PUT

Reemplaza o actualiza un recurso completo existente.

Piénsalo como: cambiar todo el pedido por uno nuevo.

Verbo DELETE

Elimina un recurso.

Piénsalo como: cancelar el pedido.

Códigos de respuesta HTTP

Números de 3 dígitos que indican qué pasó: 2xx éxito, 3xx redirección, 4xx error del cliente, 5xx error del servidor.

Piénsalo como: un semáforo de resultado — verde, amarillo, rojo por tu culpa, rojo por culpa del restaurante.

Auth. basada en sesiones

El servidor crea una sesión al iniciar sesión y la guarda en memoria/BD; le da al cliente un ID de sesión (cookie) para identificarse.

Piénsalo como: un gafete de evento que el organizador registra en su lista — sin la lista, el gafete no sirve.

Autenticación HTTP Básica

El cliente manda usuario y contraseña (codificados, no cifrados) en cada petición, en el header Authorization.

Piénsalo como: decir tu contraseña en voz alta cada vez que entras — cualquiera que escuche la copia si no hay HTTPS.

Autenticación implícita

Flujo simplificado de OAuth para apps sin backend seguro; el token vuelve directo en la URL. Hoy se considera inseguro.

Piénsalo como: gritar la llave en la calle en vez de entregarla en un sobre cerrado.

Seguridad basada en certificados

El servidor (y a veces el cliente) prueba su identidad con un certificado firmado por una autoridad de confianza — así funciona HTTPS/TLS.

Piénsalo como: una credencial oficial validada por un notario, no un gafete casero.

Auth. basada en tokens

El cliente recibe un token firmado tras el login y lo reenvía en cada petición; el servidor solo verifica la firma, sin guardar sesión.

Piénsalo como: un boleto de concierto con holograma — cualquier taquilla lo valida sin llamar a la central.

JWT JSON Web Token

El formato de token más usado hoy: header + payload (datos) + firma, codificado en Base64 y separado por puntos.

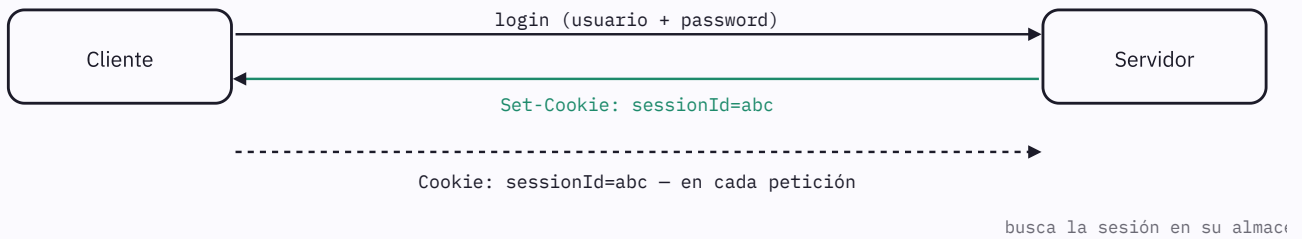
Piénsalo como: un gafete plastificado con tus datos impresos y un sello que nadie puede falsificar sin la llave.

OAuth

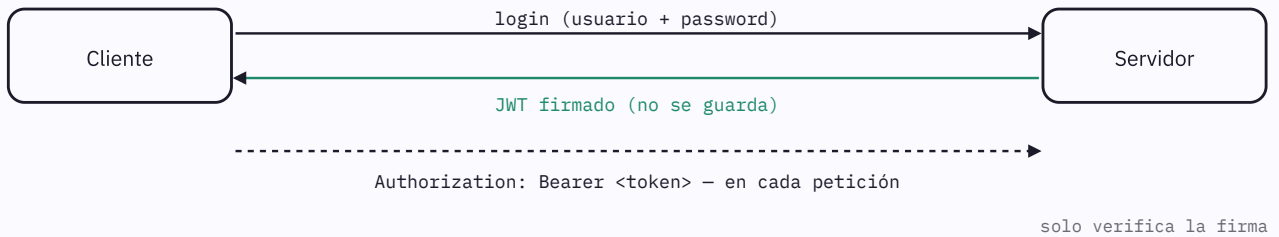
Protocolo estándar para dar acceso limitado a un recurso sin compartir la contraseña (ej. "iniciar sesión con Google").

Piénsalo como: darle al valet parking una llave que solo mueve el coche, no abre la cajuela.

Basada en sesión (con estado)



Basada en token / JWT (sin estado)



Con sesiones el servidor recuerda quién eres (con estado); con tokens/JWT el servidor no guarda nada, solo verifica que la firma sea válida (sin estado).

UNIDAD III

Desarrollo de aplicaciones del lado del servidor

Lo que se espera que sepas hacer: desarrollar aplicaciones del lado del servidor a partir del mapeo, tecnologías y servicios APIs y endpoints.

ORM mapeo objeto-relacional

Capa que traduce automáticamente entre objetos del código (clases) y filas de una base de datos relacional, sin escribir SQL a mano.

Piénsalo como: un traductor simultáneo entre el idioma de tu código y el idioma de la base de datos.

Mapeo de entidades

Definir qué clase del código corresponde a qué tabla de la base de datos.

Piénsalo como: decir "la clase Usuario = la tabla usuarios".

Mapeo de atributos

Decidir qué propiedades de un objeto sí se guardan en la base de datos (persistentes) y cuáles son temporales o calculadas (no persistentes).

Piénsalo como: en un formulario, hay campos que se archivan y notas al margen que se tiran.

Mapeo de asociaciones

Definir cómo se relacionan las tablas entre sí (uno a uno, uno a muchos, muchos a muchos) a nivel de objetos.

Piénsalo como: decir "un Cliente tiene muchos Pedidos" en vez de escribir el JOIN a mano.

Modelo de madurez de Richardson

Escala de 0 a 3 que mide qué tan RESTful es realmente una API (0 = un endpoint tipo RPC, 3 = recursos, verbos e hipermedia correctos).

Piénsalo como: un examen de qué tan bien sigues las reglas del restaurante REST, no solo si usas HTTP.

Manejo de errores (backend)

Estrategia para capturar fallas y devolver códigos/mensajes HTTP claros, en vez de que el servidor truene sin explicación.

Piénsalo como: que el mesero te diga "se acabó el platillo" en vez de desaparecer sin avisar.

CORS Cross-Origin Resource Sharing

Mecanismo del navegador que bloquea por default que una página de un dominio pida datos a una API de otro dominio, salvo que el servidor lo autorice.

Piénsalo como: el portero de un edificio que solo deja pasar visitas si están en la lista autorizada por el dueño.

Documentación de API ej. Swagger/OpenAPI

Descripción formal de qué endpoints existen, qué reciben y qué devuelven, para que otros usen la API sin adivinar.

Piénsalo como: el menú del restaurante con foto, precio e ingredientes de cada plato.

UNIDAD IV

Desarrollo de aplicaciones del lado del cliente

Lo que se espera que sepas hacer: desarrollar aplicaciones del lado del cliente a partir del diseño de interfaces y la comunicación cliente-servidor.

Desarrollo basado en componentes

Construir la interfaz como piezas reutilizables e independientes (botón, tarjeta, navbar) que se combinan; típico de React/Angular/Vue.

Piénsalo como: bloques de LEGO en vez de tallar la interfaz de una sola pieza de madera.

Diseño responsivo / adaptable

La interfaz se reacomoda sola según el tamaño de pantalla: celular, tablet o monitor.

Piénsalo como: agua que toma la forma del vaso donde la sirvas.

Pruebas de usabilidad

Verificar que personas reales puedan usar la interfaz sin confundirse, no solo que el código funcione.

Piénsalo como: ver a alguien armar un mueble solo con el instructivo, para encontrar dónde se traba.

Datos relacionales vs no relacionales

Relacional = tablas con filas/columnas fijas (SQL); no relacional = documentos o pares llave-valor más flexibles (ej. MongoDB).

Piénsalo como: relacional es un archivero con pestañas fijas; no relacional es una caja donde cada nota tiene su propio formato.

Consumo de API REST

Que el frontend haga peticiones HTTP (fetch/axios) a un backend para traer o mandar datos, en vez de tenerlos hardcodedos.

Piénsalo como: pedir comida a domicilio en vez de cocinarla tú mismo en la app.

Librerías de interfaz React, Angular, Vue

Herramientas que dan estructura y componentes listos para construir la UI más rápido y ordenado.

Piénsalo como: un kit de herramientas de carpintero en vez de tallar cada pieza a mano.

Pruebas cliente

conectividad, integridad, cifrado, seguridad

Verificar que la app se conecta al backend, que los datos no se corrompen, que viajan cifrados (HTTPS) y que no hay huecos de seguridad obvios.

Piénsalo como: revisar que el paquete llegó, que no se abrió en el camino, y que nadie lo pudo espiar.

UNIDAD V

Servidores y contenedores de aplicación en la nube

Lo que se espera que sepas hacer: gestionar entornos virtuales de desarrollo con base en tecnologías de cómputo en la nube.

Entorno virtual

Un espacio aislado en tu máquina con sus propias versiones de lenguaje/dependencias, para que un proyecto no choque con otro.

Piénsalo como: cuartos separados en la misma casa, cada quien con su propio clóset.

Contenedor Docker

Empaqueta la app + sus dependencias + un mini sistema operativo, para que corra igual en tu laptop, en el server del profe o en la nube.

Piénsalo como: un contenedor de barco — no importa qué transporte lo lleve, adentro siempre es lo mismo.

Imagen (Docker)

La plantilla congelada de un contenedor (código + dependencias) antes de ejecutarse; el contenedor es la imagen corriendo.

Piénsalo como: la imagen es la receta escrita, el contenedor es el platillo ya cocinado.

Repositorio de contenedores

ej. `Docker Hub`

Donde se suben y descargan imágenes de contenedores, como un GitHub pero de imágenes listas para correr.

Piénsalo como: una tienda de recetas ya empaquetadas.

IaaS Infraestructura como servicio

Rentas solo el hardware virtual (servidores, discos, red); tú instalas y administras todo lo demás.

Piénsalo como: rentar un terreno vacío.

PaaS Plataforma como servicio

Rentas una plataforma lista para correr tu código; el proveedor administra servidor, sistema operativo y runtime.

Piénsalo como: rentar un local de comida ya equipado con cocina — solo llevas tus recetas.

SaaS Software como servicio

Usas una aplicación completa ya hecha, sin instalar ni administrar nada (ej. Gmail).

Piénsalo como: pedir la comida ya hecha en el restaurante.

Comunicación entre microservicios

Cómo los servicios pequeños se hablan entre sí: normalmente por HTTP/REST directo o por colas de mensajes para desacoplarse.

Piénsalo como: los food trucks coordinándose por radio para no chocar pedidos.

BONUS

Stacks de la bibliografía

Las lecturas del curso apuntan a estos stacks concretos — cuando el profe los mencione, ya sabes de qué familia son.

Stack MEAN

MongoDB (BD no relacional) + Express (framework de Node) + Angular (frontend) + Node.js (runtime de JS en el servidor). Todo en JavaScript.

Stack MERN

Igual que MEAN pero con React en vez de Angular en el frontend.

Node.js

Entorno que permite correr JavaScript fuera del navegador, típicamente como servidor backend.

Express

Framework minimalista sobre Node.js para crear rutas/endpoints y servicios REST rápido.

Spring / Spring REST Java

Framework de Java para construir APIs REST y microservicios del lado del servidor; muy usado en empresas grandes.

Hibernate Java

El ORM más usado en el mundo Java; traduce clases Java a tablas de base de datos.

Kubernetes

Herramienta que administra automáticamente muchos contenedores a la vez: los crea, reinicia si truenan, y los escala.

Piénsalo como: el jefe de flota que decide cuántos food trucks mandar según la demanda.

Webpack

Empaqueta todo el código frontend (JS, CSS, imágenes) en archivos optimizados para el navegador.

TypeScript

JavaScript con tipos — avisa de errores antes de correr el código, no solo cuando ya truena.